

METHODOLOGY OVERVIEW

Factory.ai Missions Architecture

Building software with a multi-role pipeline
and externalized state

When AI agents — or human teams — build software in a single long conversation:

- Context drifts and gets forgotten across turns
- The same person who writes the code also reviews it — confirmation bias
- "Done" means different things to different people
- Scope creeps because everything feels related to the current task
- Decisions made in week 1 are re-discovered from scratch in week 6

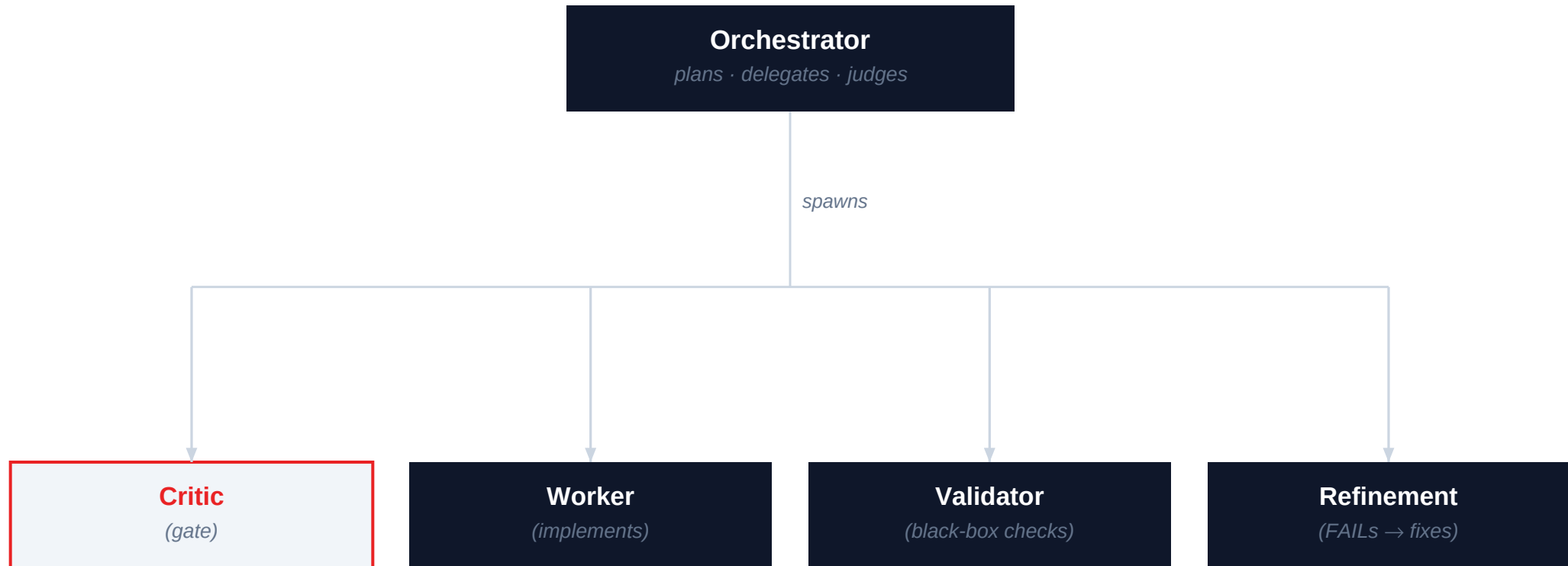
Missions Architecture fixes this with role separation, file-based state, and TDD at two levels.

**Turn "build a feature" into a pipeline
of specialized roles, each running in a
fresh context, communicating only through files.**

The Four Roles

Each role runs in a fresh Claude session — no shared memory, only files

4 / 22



Role	Does	Must NOT do
Orchestrator	Picks next feature, runs gates, spawns roles, processes results	Write code
Critic	Reviews specs / architecture / decomposition before any work	Edit the artifact it reviews
Worker	Implements ONE feature with TDD, scope-disciplined	Refactor adjacent code, mark "done"
Validator	Exercises the system as a real user, PASS/FAIL/BLOCKED per assertion	Read worker output, propose fixes
Refinement	Converts FAILs into minimum-scope fix features	Implement

State Lives in Files, Not in Memory

Three sacred files — the only persistent state in the system

File	Purpose	Writer	Reader
mission/ validation-contract.md	Black-box assertions the system must satisfy. Each has a stable ID like auth.3	Author + Critic feedback	Validator, Critic
mission/ features.json	Backlog: id, scope, dependencies, status	Orchestrator + Worker (status field only)	Everyone
mission/ knowledge-base.md	Append-only log of decisions, gotchas, learnings	Worker + Validator	Worker (so mistakes are not repeated)

Plus per-run logs in `mission/runs/`

Each assertion names a concrete input and a concrete, observable result:

```
auth.3: POST /users body={"email":"a@b.com"} when an account with  
that email already exists → HTTP 409 {"code":"USER_EXISTS"}
```

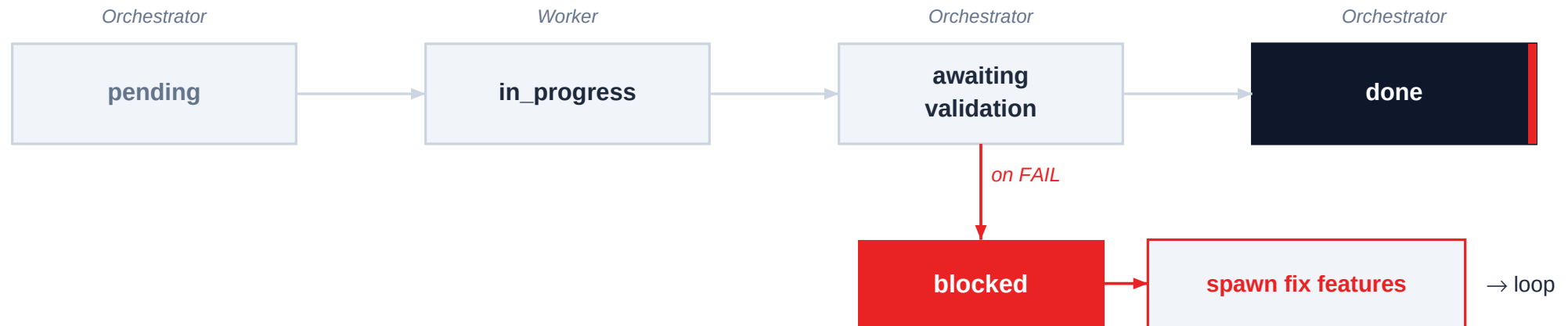
- **Concrete input** — Exact method, endpoint, and request body are specified.
- **Concrete observable result** — Status code and response body — both externally verifiable.
- **Implementation-agnostic** — No library names, no internal mechanism, no framework details.
- **Exercisable without source** — Anyone with a running system can verify this assertion.

The contract is written before the feature. Features reference assertions via `validation_refs`.

Feature Lifecycle

Status transitions are owned by specific roles only

8 / 22




```
{
  "id": "F03a",
  "title": "User can sign up via email + password",
  "phase": 1,
  "status": "pending",
  "depends_on": ["F00b"],
  "scope": "Adds POST /api/users. Validates email + password length >=8.
           Returns 201 {id, email}. Returns 409 if email already exists.",
  "validation_refs": ["auth.1", "auth.3"]
}
```

- **scope** — is fact, not promise — it describes the exact boundary of this feature
- **validation_refs** — link directly to assertions in the validation contract
- **depends_on** — prevents the Orchestrator from scheduling work out of order

The Critic Gate

Runs before any Worker spawns — in two sequential phases

10 / 22

Mechanical Checks

deterministic · cannot be overridden

- Run by a Node script; exit code non-zero on failure
- Catch typos, missing fields, dependency cycles, orphan references
- Zero LLM cost — must pass before the model sees any artifact

Judgment Checks

LLM · must produce verbatim concrete suggestions

- Every "needs-work" finding must include the exact rewrite
- "Too vague" without a concrete fix is forbidden
- No finding is accepted without a specific, actionable replacement

Mechanical checks always run first. A failing mechanical check blocks the LLM phase entirely.

Phase	Reviews	When it gates
A	validation-contract.md	Any feature with non-empty validation_refs
B	Architecture section of CLAUDE.md (with 3 adversarial counterexamples tailored to this project's stack)	First feature of every new phase, or any feature that changes architecture
C	features.json decomposition	Any feature added or modified since the last passing report

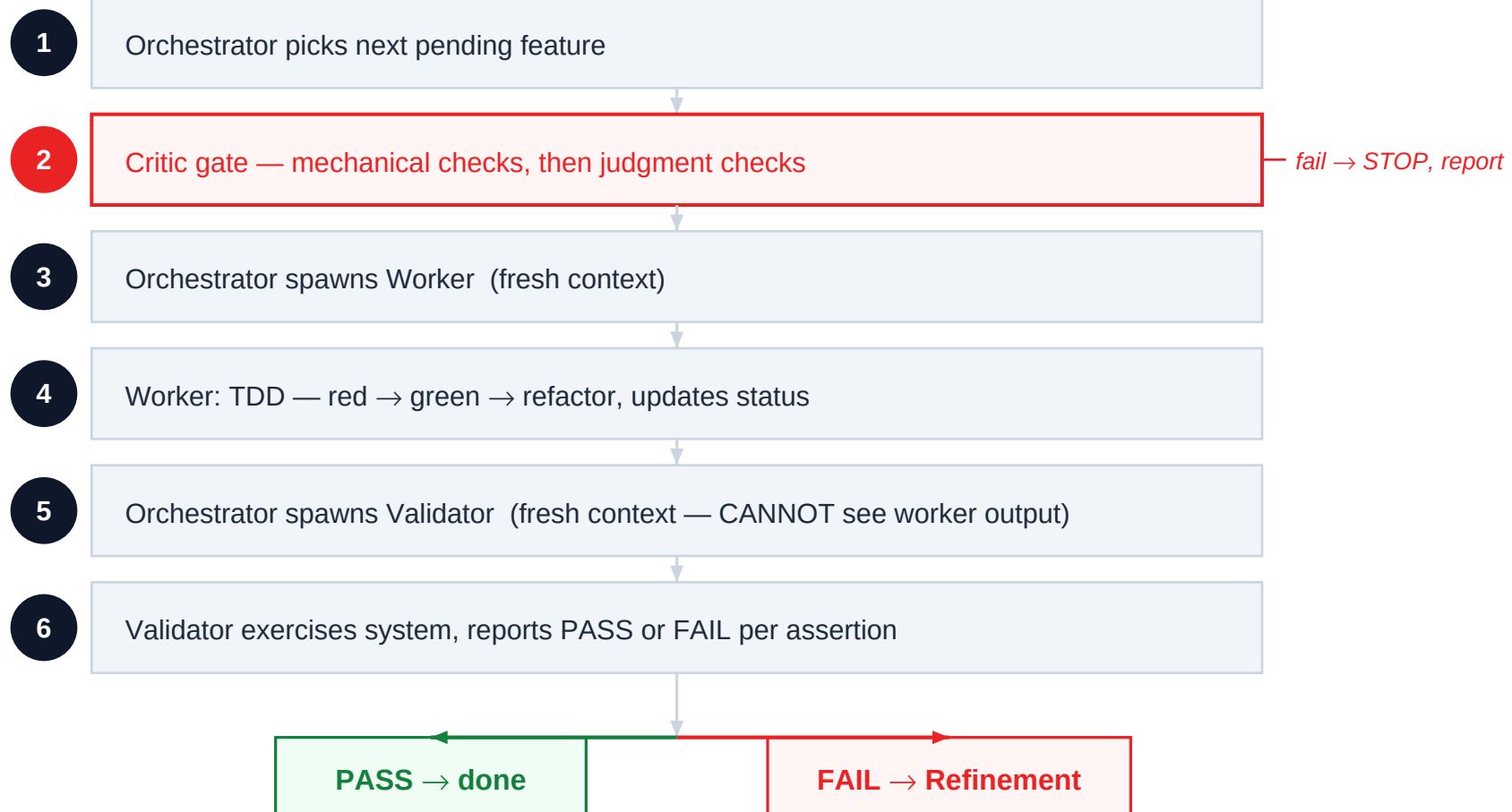
Phases can be run individually or all three at once.

ID	What it catches
J-S1 Testable	Assertion lacks a concrete input or a concrete, observable result
J-S2 Falsifiable	No experiment could prove it wrong (e.g. "system is fast")
J-S3 Implementation-agnostic	Prescribes a library, table, or middleware (e.g. "JWT with exp claim")
J-S4 Independent	Meta-assertions that depend on other assertions ("all of the above hold")
J-S5 Coverage	Only happy-path assertions — no failure modes or edge cases tested
J-S6 Stable language	"Recently", "the new endpoint", "after the migration" — language that rots fast

One Round, End to End

The complete pipeline for a single feature

13 / 22



Why the Validator Cannot See the Worker

The most important constraint in the entire methodology

If the validator reads the worker's diff, confirmation bias is inevitable.

"I think it looks right" creeps in — and it is almost always wrong in subtle ways.

Validator CAN read

- The validation contract
- Current codebase
- Knowledge base
- Logs, DB, screenshots

Validator CANNOT read

- Worker's output or diff
- The PR or any "what was done" report
- Any summary of worker activity

Evidence is mandatory. PASS without concrete evidence is a guess, not a pass.

Level 1 — Worker

unit / integration tests

- Tests cover every assertion in validation_refs
- Red first — write the failing test before any code
- Minimum code to reach green
- Refactor only after green

Level 2 — Validator

end-to-end exercises

- Completely independent of the worker's own tests
- Sends real HTTP requests, hits real endpoints
- If the only evidence is the worker's test, Validator writes an independent exercise
- Collects concrete evidence per assertion

This double-loop catches "tests pass but the feature does not actually work."

When Validation Fails

Refinement converts each FAIL into a minimum-scope fix feature

16 / 22

```
{
  "id": "F03a-fix-1",
  "title": "Handle duplicate email case",
  "depends_on": ["F03a"],
  "scope": "Add INSERT conflict handling to /api/users. Return 409 with
           {code: USER_EXISTS} when email collides.",
  "validation_refs": ["auth.3"],
  "fixes": "F03a",
  "addresses": ["auth.3"]
}
```

- Original feature stays blocked — the failure is preserved for history
- Fix feature runs through the same full pipeline: Critic gate, Worker, Validator
- No exceptions: small fixes do not skip validation

Greenfield

init-mission skill · empty repo

- Asks 6 questions: name, description, architecture, languages, hosting, regulated domain
- Creates mission/ folder, CLAUDE.md, README.md, .gitignore
- Pre-seeds all files with templates

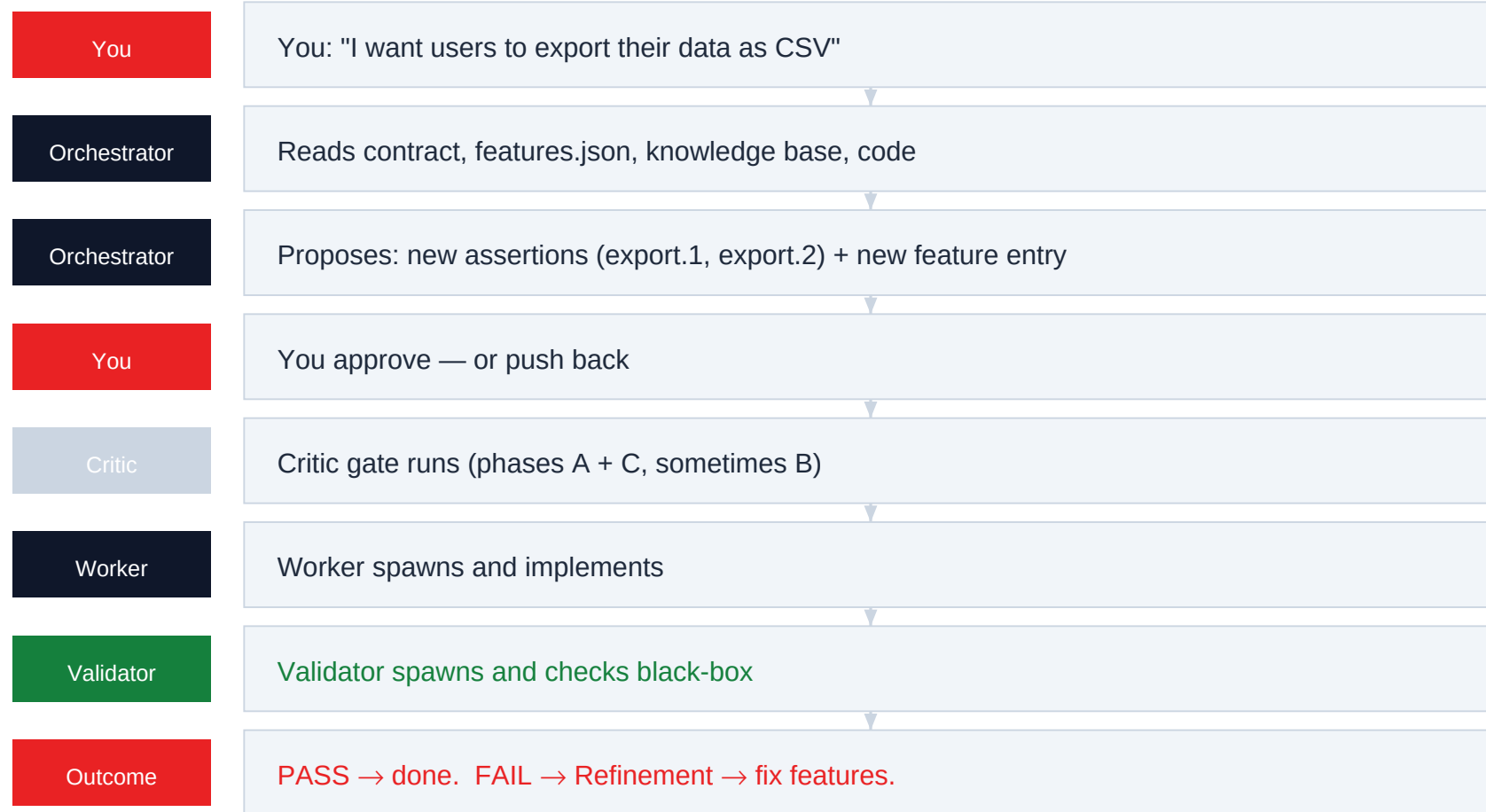
Brownfield

existing repo

- Auto-detects stack from package.json, pyproject.toml, Cargo.toml, etc.
- Detects owner from git config, hosting from deploy configs
- Inventories existing HTTP routes, CLI commands, tests — seeds contract with [CONFIRM] drafts
- Appends to CLAUDE.md / README / .gitignore — never overwrites
- Only asks for what could not be inferred
- Always asks: regulated domain?

Adding a New Feature to an Existing Project

18 / 22



You only touch steps 1 and 4. Everything else is automated.

Constraint	Prevents
State in files	Context drift, conversation amnesia
Fresh context per role	Confirmation bias, role conflation
Validator blind to worker output	"Looks right to me" — the #1 self-review failure
Critic gate before any Worker	Wasting tokens implementing against a bad spec
Mechanical checks first	Burning LLM tokens on trivially fixable issues
Assertions → tests mapping	"What we said" drifting from "what we built"
Scope discipline	Feature creep, bundled PRs
Append-only knowledge base	Re-discovering the same gotcha twice
Contract written before feature	Validator having nothing objective to grade against

Skip it for:

- One-off refactors and obvious fixes — a single round of work does not justify the overhead
- Exploratory phases where you do not yet know what to build — talk to a human first
- Single-author throwaway scripts

Use it when:

- The project will live long and be touched across multiple sessions or by multiple people
- Correctness matters and regressions are expensive
- You want machine-verifiable done

Key Takeaways

21 / 22

1

Roles, not personas

Each role has a strict job and strict prohibitions. No blurring.

2

Files are sacred

Everything important lives outside any single conversation.

3

Contract comes first

No assertion → no feature → no validation. Always in that order.

4

Black-box validation

The Validator never sees the Worker. Confirmation bias eliminated by design.

5

The Critic gate

Bad specs and decompositions caught before a single line of code is written.

6

TDD at two levels

Worker tests + Validator exercises together catch "passes tests, does not work".

Missions Architecture is opinionated on purpose. Each constraint exists because someone learned the hard way that the alternative produces software that drifts, regresses, or quietly stops matching its spec.

You give up some velocity on day 1.
You get it back tenfold by month 3.